

Enhancing Event-Frame Feature Tracking with Siamese FPN

(Supplementary Material)

Andreas Fotakis[✉] and Emmanouil Psarakis[✉]

Department of Computer Engineering and Informatics, University of Patras, 26504
Rion-Patras, Greece
`{st1084674,psarakis}@ceid.upatras.gr`

1 Event representations

In order to achieve our goal let us specify the event representations we are going to use.

To this end, we split the set $\mathcal{E}_q$ (please see Eq. (4) of the paper) into M consecutive temporal windows of duration T each, that is:

$$\mathcal{E}_q = \bigcup_{m=1}^M \mathcal{E}_m, \text{ with } \mathcal{E}_m = \left\{ \mathbf{e}_k \in \mathcal{E}_q \mid t_{j-1} + (m-1)T \leq t_k \leq t_{j-1} + mT \right\}.$$

Then, each window is further partitioned into N_B equal temporal bins of duration $\Delta T = \frac{T}{N_B}$ each, and we define:

$$\begin{aligned} \mathcal{E}_m &= \bigcup_{i=1}^{N_B} \mathcal{E}_{m,i} \\ \mathcal{E}_{m,i} &= \left\{ \mathbf{e}_k \in \mathcal{E}_m \mid t_{j-1} + (m-1)T + (i-1)\Delta T \leq t_k \leq t_{j-1} + (m-1)T + i\Delta T \right\} \end{aligned} \quad (1)$$

Finally, following the SBT-Max representation [4] we split each bin by polarity, i.e.:

$$\mathcal{E}_{m,i} = \bigcup_{p \in \{0,1\}} \mathcal{E}_{m,i}^p, \quad \mathcal{E}_{m,i}^p = \left\{ \mathbf{e}_k \in \mathcal{E}_{m,i} \mid p_k = p \right\}, \quad (2)$$

and create a tensor with $2N_B$ channels (two polarities per bin). Using the channel index $j = pN_B + i \in \{1, \dots, 2N_B\}$, the value at pixel $\mathbf{n}$ is defined as:

$$V_{j,l}^m(\mathbf{n}) = \frac{1}{T} \max_{\substack{\mathbf{e}_k \in \mathcal{E}_{m,i}^p \\ \mathbf{n}_k = \mathbf{n}}} \left(0, t_k - t_{j-1} - (m-1)T \right), \quad (3)$$

that is, for the pixel $\mathbf{n}$ of the m -th window in the i -th bin with polarity p , we take the most recent event timestamp in $\mathcal{E}_{m,i}^p$ occurring at $\mathbf{n}$, measure its offset from the window start $t_{j-1}(m-1)T$ and normalize it by T . If such an event does not exist, we set it to zero.

2 Evaluation metrics

In order to evaluate the proposed method and compare its performance against its rivals, let $\tilde{\mathbf{x}}_{f_l}(t; \Theta^*)$ denote the predicted trajectory of feature f_l , linearly interpolated at the ground-truth timestamps $t \in [t_0, T_l]$, with T_l denoting the duration of the l -th feature's trajectory, and:

$$\varepsilon_{f_l}^*(t) = \|\mathbf{x}_{f_l}^{GT}(t) - \tilde{\mathbf{x}}_{f_l}(t; \Theta^*)\|_2 \quad (4)$$

the L_2 norm of the tracking error at time t . Let us also consider that for a pixel threshold $r > 0$, $r \in \mathcal{N} = \{1, \dots, N\}$, $t_{f_l}^*(r)$ denotes the last timestamp in the interval $[t_0, T_l]$ before the inequality $\varepsilon_{f_l}^*(\tau) \leq r$ is violated for the first time. Then, given a termination threshold $\tau > 0$, let

$$t_{f_l}(r^*) = \arg \min_{r \in \{1, \dots, K_l\}} \{\varepsilon_{f_l}(\tau) > \tau\} \quad (5)$$

If no such violation occurs, we set $t_{f_l}^*(r) = T_l$. Then, the *relative feature age* of feature f_l at threshold r is defined as follows:

$$\text{FA}_{f_l}(r) = \frac{t_{f_l}^*(r) - t_0}{T_l - t_0}. \quad (6)$$

Considering now only the *alive* features $\mathcal{F}_{\text{alive}}(r) = \{f_l \in \mathcal{F} \mid \text{FA}_{f_l}(r) > 0\}$, at threshold r , the corresponding *feature age* (FA) can be defined as follows:

$$\text{FA}(r) = \frac{1}{|\mathcal{F}_{\text{alive}}(r)|} \sum_{f_l \in \mathcal{F}_{\text{alive}}(r)} \text{FA}_{f_l}(r), \quad (7)$$

with $|\mathcal{A}|$ denoting the cardinality of set $\mathcal{A}$. Note that by dividing the cardinality of set $\mathcal{F}_{\text{alive}}(r)$ with the cardinality of set $\mathcal{F}$, the *inlier ratio* at threshold r can be defined, i.e.,:

$$\text{IR}(r) = \frac{|\mathcal{F}_{\text{alive}}(r)|}{N_f} \quad (8)$$

that can be used to define the *expected feature age* (EFA) at the same threshold as follows:

$$\text{EFA}(r) = \text{IR}(r) \text{FA}(r) = \frac{1}{N_f} \sum_{f_l \in \mathcal{F}_{\text{alive}}} \text{FA}_{f_l}(r). \quad (9)$$

Having defined the feature aging $\text{FA}(r)$ and its expected counterpart $\text{EFA}(r)$, by evaluating them over all the elements of set $\mathcal{N}$, we can define the following quantities:

$$\text{FA} = \frac{1}{|\mathcal{N}|} \sum_{r \in \mathcal{N}} \text{FA}(r), \text{ and } \text{EFA} = \frac{1}{|\mathcal{N}|} \sum_{r \in \mathcal{N}} \text{EFA}(r), \quad (10)$$

that we are going to use in our evaluation.

Table 1: Efficiency comparison between DDFT and the proposed model on the EDS and EC datasets. Lower values are better for the number of parameters, inference time, forward-pass latency, GFLOPs, and peak memory, while higher FPS is better.

Method	Dataset	Params. (M) ↓	Inf. time (s) ↓	Fwd. (ms) ↓	FPS ↑	GFLOPs ↓	Peak mem. (MiB) ↓
DDFT [4]	EDS	35.0	8.55	17.47	60.50	240.04	523.92
Ours	EDS	20.0	8.41	17.22	61.36	245.35	411.13
DDFT [4]	EC	35.0	4.45	12.24	86.61	189.62	464.44
Ours	EC	20.0	4.41	12.14	87.38	193.82	351.32

3 Ablation study

3.1 Model Efficiency

To evaluate the practical efficiency of the proposed architecture, we compare it with the original DDFT model [4] in terms of end-to-end inference time, forward-pass latency, throughput, computational complexity, and peak GPU memory consumption. Both models are evaluated under the same hardware, software, and evaluation settings. The reported values are averaged over all sequences of each real-world dataset.

As reported in Table 1, the proposed model provides comparable or slightly improved runtime efficiency while requiring substantially less peak GPU memory. On EDS, the proposed model increases the average throughput from 60.50 to 61.36 FPS, corresponding to an improvement of approximately 1.42%. At the same time, the forward-pass latency decreases from 17.47 to 17.22 ms, while peak GPU memory consumption decreases from 523.92 to 411.13 MiB, representing a reduction of approximately 21.53%.

On EC, the throughput increases from 86.61 to 87.38 FPS, corresponding to an improvement of approximately 0.89%, and the forward-pass latency decreases from 12.24 to 12.14 ms. More importantly, peak GPU memory consumption decreases from 464.44 to 351.32 MiB, corresponding to a reduction of approximately 24.36%.

The proposed model has slightly higher computational complexity, increasing from 240.04 to 245.35 GFLOPs on EDS and from 189.62 to 193.82 GFLOPs on EC, an increase of approximately 2.21% in both cases. This increase is caused by the modified convolutional refinement layers. Weight sharing primarily reduces the number of distinct learnable encoder parameters and their associated memory requirements; it does not eliminate the computation of either modality because the shared FPN must still be applied separately to the event and frame inputs. Therefore, a direct reduction in FLOPs is not necessarily expected.

Overall, the principal lightweight advantage of the proposed Siamese formulation is its removal of redundant event and frame encoder parameters, which translates into a considerably smaller GPU memory footprint without sacrificing runtime performance.

Table 2: Detailed per-sequence performance comparison on the EDS (top) and EC (bottom) datasets. Bold values indicate the best result among event-frame methods. Higher values are better.

Method	Frames	Average		Peanuts Light		Rocket Earth		Ziggy Arena		Peanuts Running	
		FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$
EKLT [1]	✓	0.325	0.205	0.284	0.260	0.425	0.175	0.419	0.231	0.171	0.153
DDFT [4]	✓	0.576	0.472	0.447	0.420	0.648	0.291	0.748	0.746	0.460	0.428
FE-TAP [3]	✓	0.676	0.589	0.549	0.517	0.538	0.246	0.849	0.844	0.769	0.749
Ours	✓	0.616	0.521	0.550	0.517	0.664	0.329	0.682	0.681	0.569	0.554
ETAP [2]	×	0.705	0.598	0.529	0.500	0.705	0.336	0.839	0.838	0.746	0.717

Method	Frames	Average		Shapes Trans.		Shapes Rot.		Shapes 6-DoF		Boxes Trans.		Boxes Rot.	
		FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$	FA $\uparrow$	EFA $\uparrow$
EKLT [1]	✓	0.811	0.775	0.839	0.740	0.833	0.806	0.817	0.696	0.682	0.644	0.883	0.865
DDFT [4]	✓	0.825	0.818	0.861	0.865	0.797	0.793	0.899	0.882	0.872	0.869	0.695	0.691
FE-TAP [3]	✓	0.844	0.838	0.931	0.929	0.815	0.813	0.879	0.860	0.731	0.728	0.862	0.861
Ours	✓	0.866	0.858	0.885	0.878	0.828	0.824	0.884	0.866	0.890	0.884	0.846	0.838
ETAP [2]	×	0.888	0.883	0.910	0.904	0.867	0.865	0.904	0.886	0.866	0.864	0.896	0.893

3.2 Detailed Comparison

The performance of all compared methods on the individual EDS and EC sequences is reported in Table 2. The proposed tracker consistently outperforms the other event-frame methods on the EC sequences, whereas its lower aggregate performance on EDS is mainly caused by the challenging *Ziggy Arena* and *Peanuts Running* sequences. These sequences contain difficult low-light conditions, abrupt exposure variations, occlusions, and rapid independent object motion, as also noted in previous studies [2–4].

One factor contributing to the performance difference on *Peanuts Running* is the temporal availability of frame information. The camera frame rate is lower than the tracker unrolling frequency, and our method therefore continues to match the incoming event representation against a fixed reference frame patch. In contrast, FE-TAP uses reconstructed frames at intermediate timestamps, providing more frequently updated appearance information and potentially improving its robustness under rapid motion and substantial appearance changes. Event-based vision algorithms are fundamentally valued for their microsecond-level temporal resolution and low latency and the requirement for reconstructed frames limits these advantages. Event-only methods are similarly not constrained by the conventional frame rate and can update their predictions directly from the high-rate event stream.

Figure 1 shows the trajectories of the proposed tracker on some representative EDS (left) and EC (right) sequences.

Tracking performance is further compromised by limitations in the feature-initialization and selection pipeline. In sequences characterized by significant object or camera motion, features often migrate toward image boundaries or exit the field of view. Such tracks are irrecoverable and artificially lower the reported FA and EFA metrics. Specific instances from Figure 1 include:

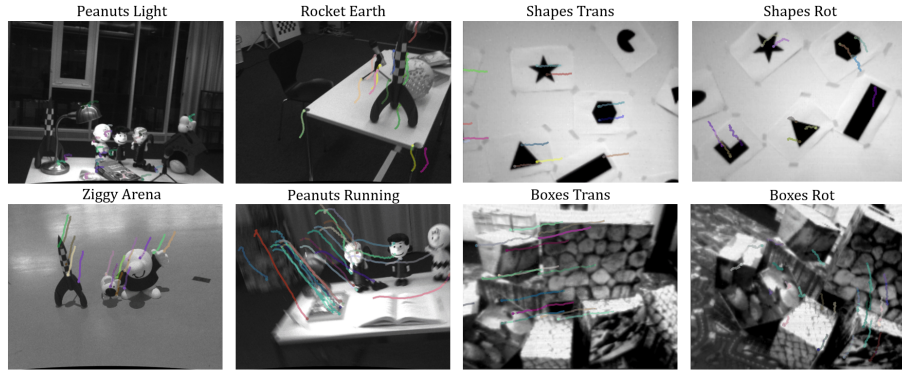


Fig. 1: Qualitative visualization of feature tracks on representative EC and EDS sequences

- *Rocket Earth*: An initialized feature on the lamp moves out of frame.
- *Shapes Translation*: The selected corners of a shape leave the field of view.
- *Peanuts Running*: A tracked corner of the desk becomes occluded.

Thus, the spatial distribution of initialized features and their lifespan of visibility remain critical factors influencing performance on challenging datasets.

References

1. Gehrig, D., Rebecq, H., Gallego, G., Scaramuzza, D.: Asynchronous, Photometric Feature Tracking Using Events and Frames, pp. 766–781. Springer International Publishing (2018). https://doi.org/10.1007/978-3-030-01258-8_46
2. Hamann, F., Gehrig, D., Febryanto, F., Daniilidis, K., Gallego, G.: Etap: Event-based tracking of any point (2025)
3. Liu, J., Wang, B., Tan, Z., Zhang, J., Shen, H., Hu, D.: Tracking any point with frame-event fusion network at high frame rate (2024)
4. Messikommer, N., Fang, C., Gehrig, M., Cioffi, G., Scaramuzza, D.: Data-driven feature tracking for event cameras with and without frames. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **47**(5), 3706–3717 (May 2025). <https://doi.org/10.1109/tpami.2025.3536016>